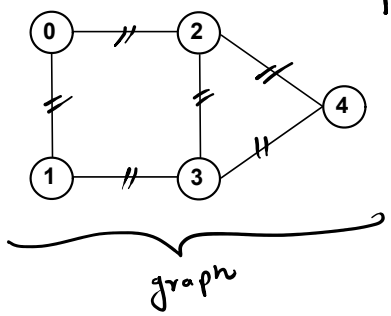
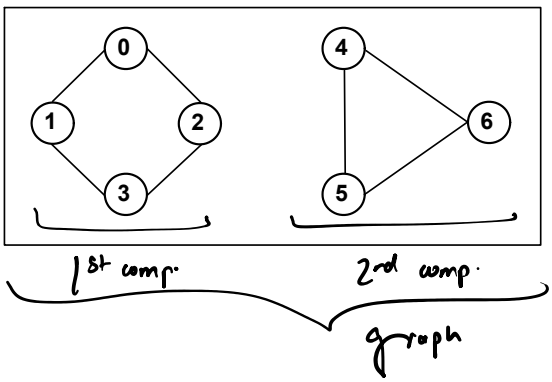
Graphs

It is a **non-linear** data-structure that can be thought of as a collections of **nodes** (or vertices) and **edges**.



V = { 0, 1, 2, 3, 4 }

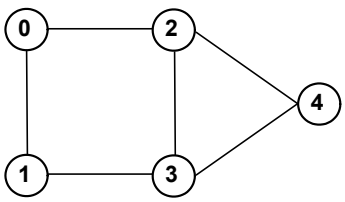
E = { {0, 1},
{0, 2},
{1, 3},
{2, 3},
{2, 4},
{3, 4} }



V = { 0, 1, 2, 3, 4, 5, 6 }

E = { {0, 1},
{0, 2},
{1, 3},
{2, 3},
{4, 5},
{4, 6},
{5, 6} }

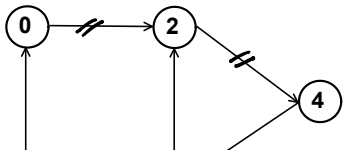
Formally, a graph is a **pair of sets V** and **E** where **V** is a **non-empty** set of **vertices** and **E** is a set of **edges** such that each **edge is a pair of vertices**.



V = { 0, 1, 2, 3, 4 }

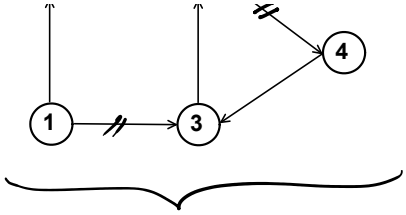
E = { {0, 1},
{0, 2},
{1, 3},
{2, 3},
{2, 4},
{3, 4} }

In a **directed graph**, each edge is an **ordered** pair of vertices.



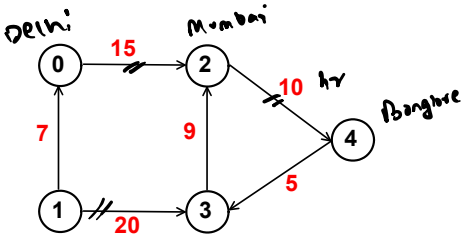
V = { 0, 1, 2, 3, 4 }

E = { (0, 2),
(1, 0),
(1, 3),



E = { (0, 2),
(1, 0),
(1, 3),
(2, 4),
(3, 2),
(4, 3) }

In a **weighted graph**, each edge is assigned a weight.



V = { 0, 1, 2, 3, 4 }

E = { (0, 2, 15),
(1, 0, 7),
(1, 3, 20),
(2, 4, 10),
(3, 2, 9),
(4, 3, 5) }

Wgt Unwgt

Dir . .

UnDir - .

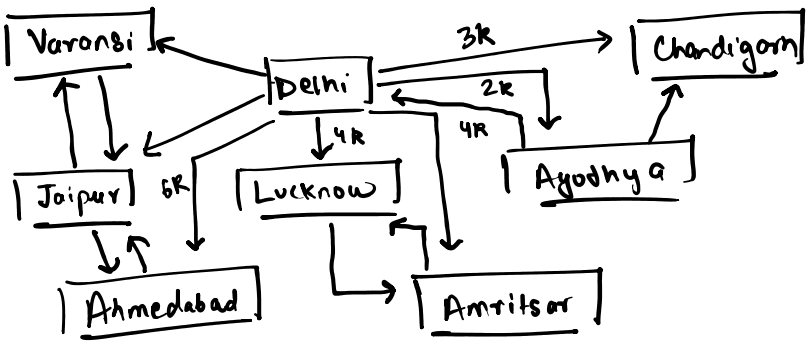
Applications of Graphs

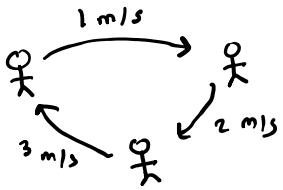
1. **Social Networking Sites**

- **Users :** can be thought as nodes in a graph
- **Relationships :** can be thought as edges between graph nodes.

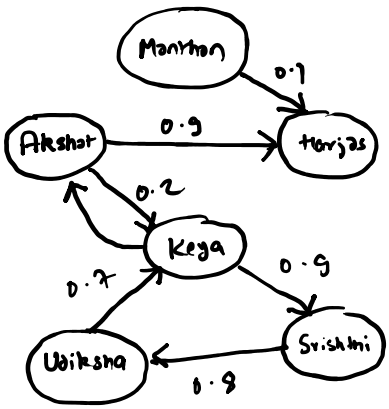
2. **Maps (of a city)**

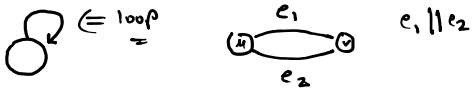
- **Checkpoints :** can be thought as nodes in a graph
- **Roads :** can be thought as edges between graph nodes.





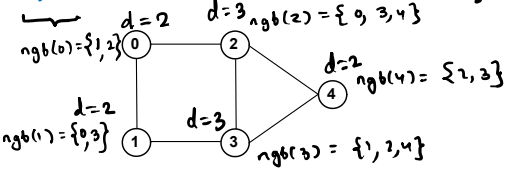
0 —— 1

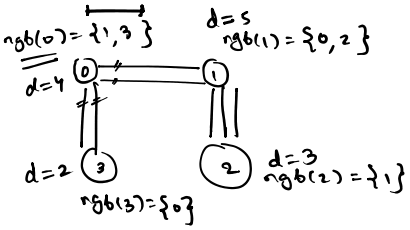


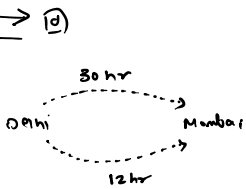


Graph Terminology

- In an undirected graph, if there exists an edge between two vertices then they are said to be **adjacent** to each other.

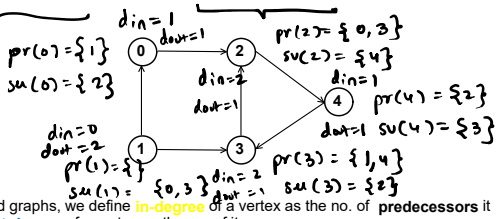






- In an undirected graph, we define **degree** of a vertex as the number of vertices adjacent to it.

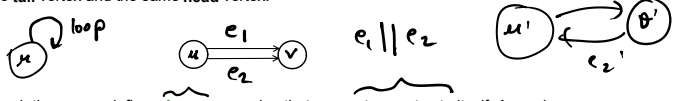
- In a directed graph, if there exists an directed edge **u** → **v** then we say that vertex **u** is **predecessor** of vertex **v** or vertex **v** is **successor** of vertex **u**.



- For directed graphs, we define **in-degree** of a vertex as the no. of **predecessors** it has and **out-degree** of a vertex as the no. of its **successors**.

- In an undirected graph, two or more edges are said to be **parallel edges** if they are **incident** on the same two vertices.

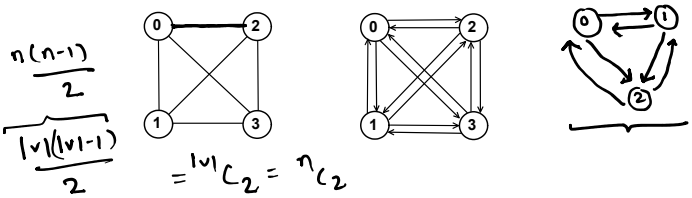
- In a directed graph, two or more edges are said to be **parallel edges** if they have the same **tail** vertex and the same **head** vertex.



- In graph theory, we define a **loop** as an edge that connects a vertex to itself. A graph that doesn't have parallel edges and loops is also known as a **simple graph.**

Minimum and Maximum Number of Edges in Graph

- The **minimum** no. of edges in a graph is **zero**, such a graph is known as **empty graph**.
- For **maximum** no. of edges in a graph, **each vertex** of the graph must be **connected** with every other graph vertex.



Graph Representation

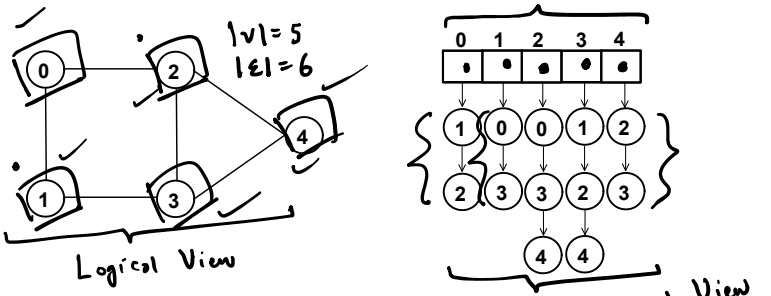
There are two key strategies that can be used to represent the graph data structure

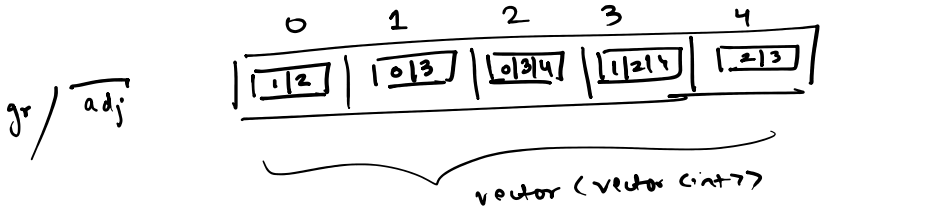
- Adjacency List
- Adjacency Matrix

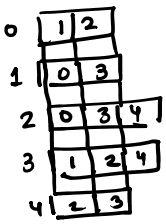
At a high-level, in each of these strategies, we internally implement the graphs using arrays which is indexed by the graph vertices.

Adjacency List

To represent a graph that contains | V | vertices and | E | edges using adjacency list, you've to create an **array** of size | V | such that at each index of the array you store a list of **neighbors** that correspond to the vertex which has been mapped to that index.



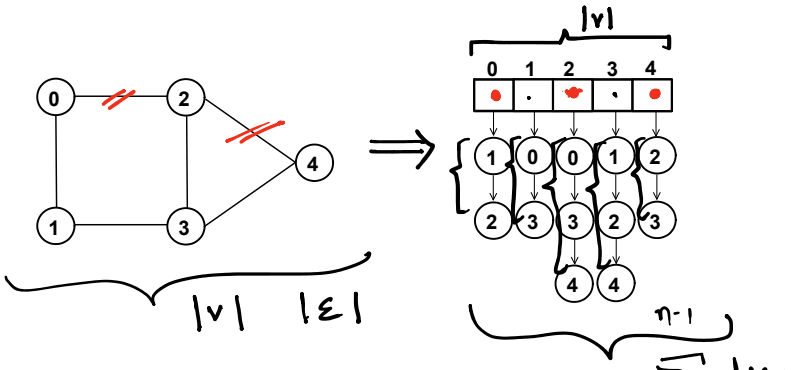


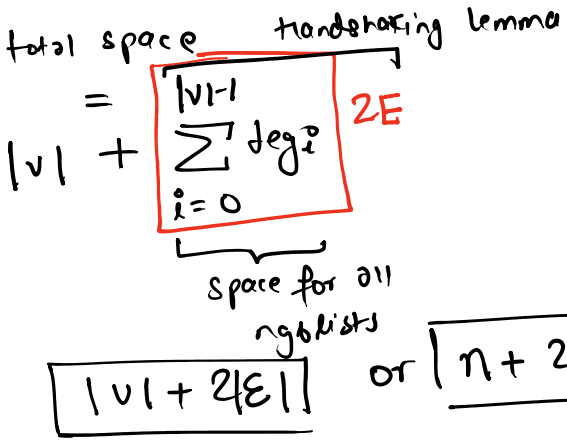


If you want to make sure nghlist is always sorted use a vector<set<int>>

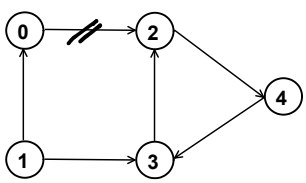
Space Complexity

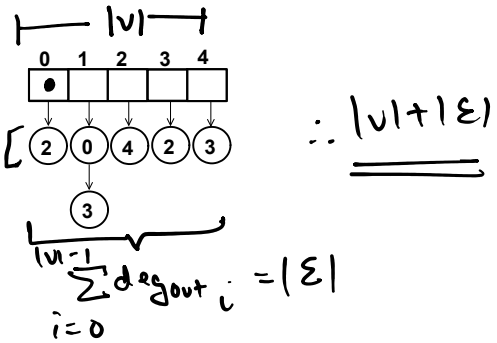
Assume you are given an **undirected** graph that contains | V | vertices and | E | edges.





Assume you are given an **directed** graph that contains | V | vertices and | E | edges.





Graph Operations

Assume you are given an graph that contains | V | vertices and | E | edges.

- List all the **neighbors** of a vertex 'u'

time $\propto O(deg(u))$

- **Add an edge** between vertex 'u' and 'v'

vec<vec>: adj[u].pb(v) — const

or

vec<set>: adj[u].insert(v) — $O(\log(deg(u)))$

- **Check** if an edge exists between vertex 'u' and vertex 'v'

go to nghlist of u and search for v or go to nghlist of v and search for u

better option : search in smaller nghlist

vector<vector> $\Rightarrow$ time $\propto O(\min(deg(u), deg(v)))$

vector<set> $\Rightarrow$ time $\propto O(\log(\min(deg(u), deg(v))))$

- **Delete** an edge between vertex 'u' and 'v'

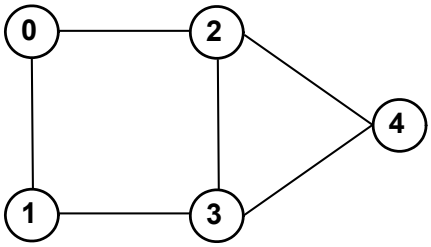
vec<vec>: time : $O(deg(u) + deg(v))$ — can be optimized to $O(\max(deg(u), deg(v)))$

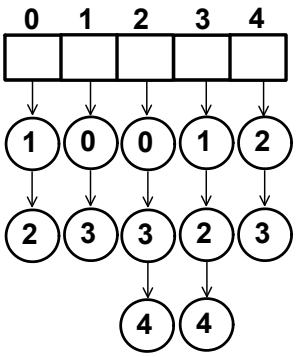
vec<set>: time : $O(\log(deg(u)) + \log(deg(v)))$

go to nghlist of u and del. v
then go to nghlist of v and del. u

Adjacency List

To represent a graph that contains | V | vertices and | E | edges using adjacency list, you've to create an **array** of size | V | such that at each index of the array you store a list of **neighbors** that correspond to the vertex which has been mapped to that index.

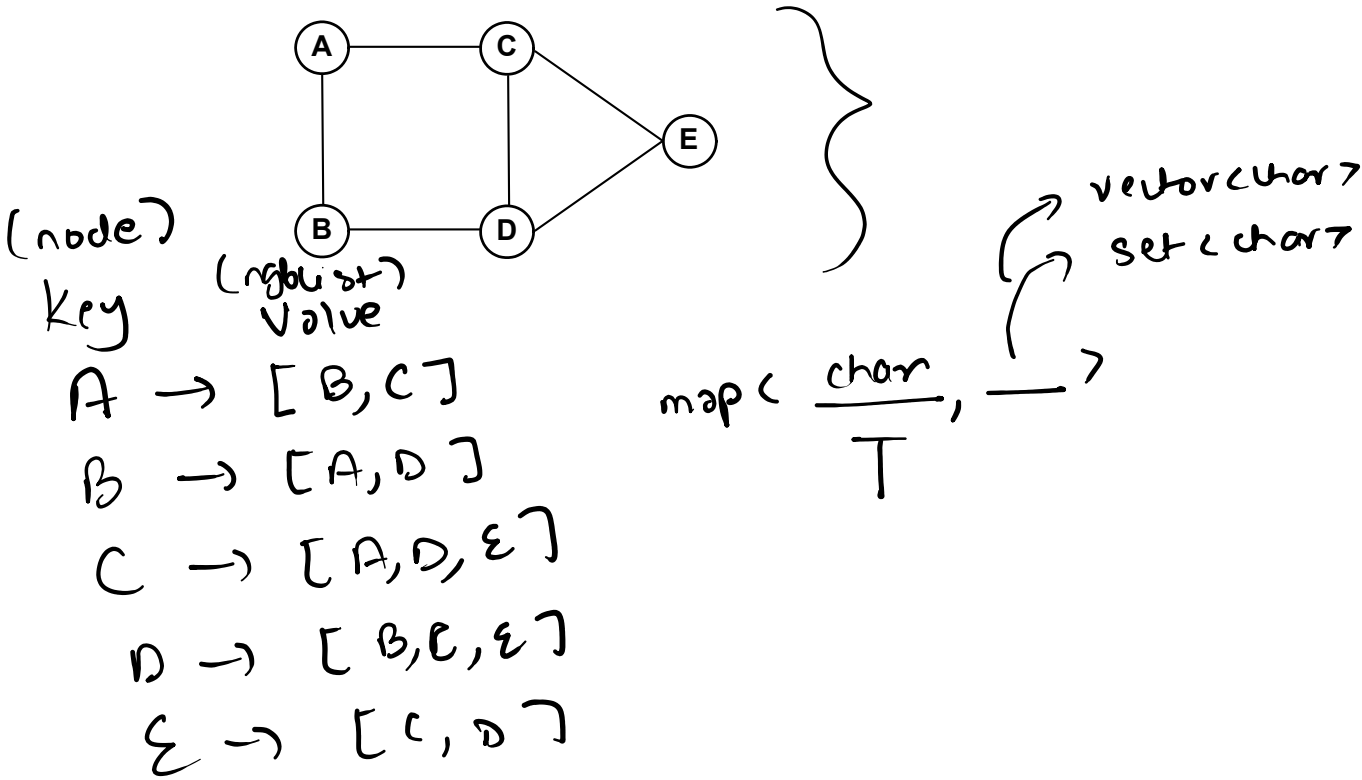




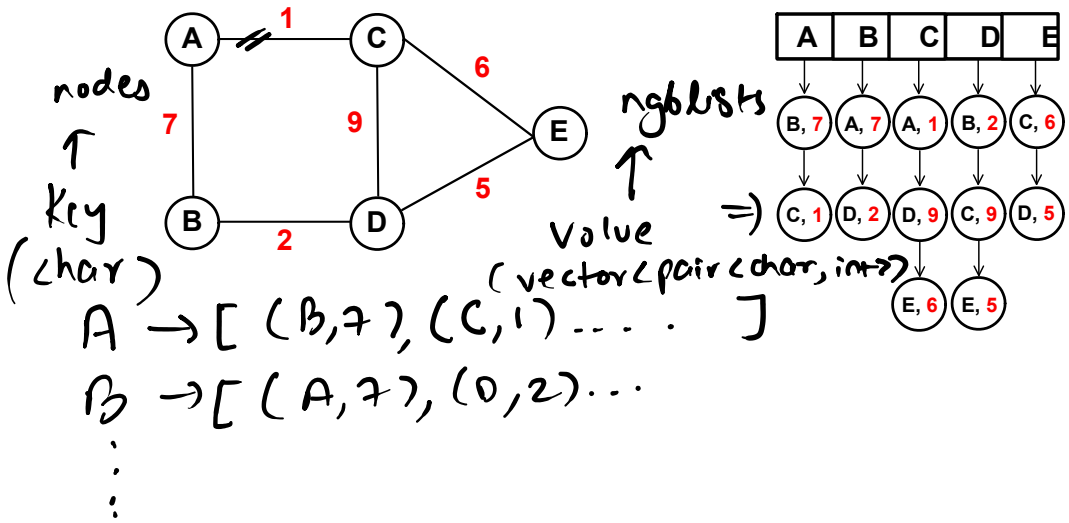
Drawbacks

Our current implementation fails when labels assigned to vertices are

- integer values that exceed | V | -1
- non-integer values

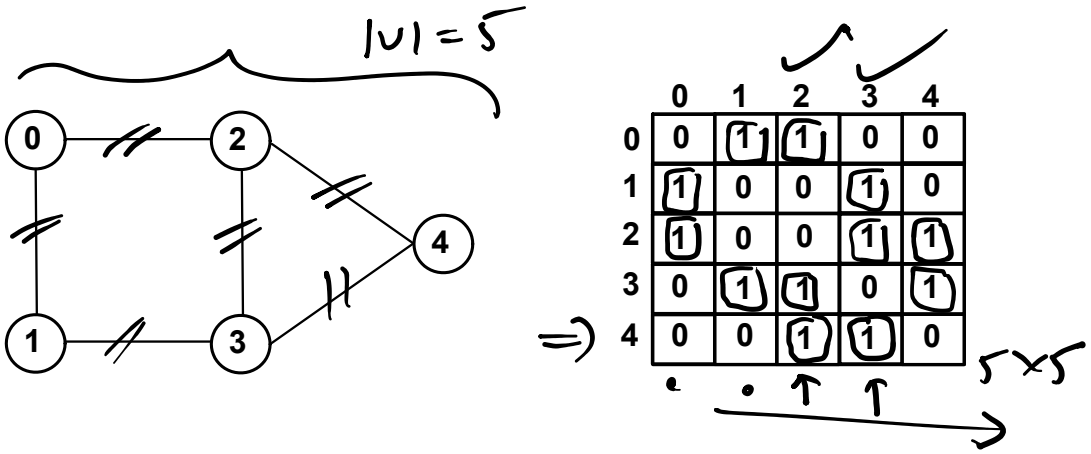


Weighted Graphs



Adjacency Matrix

To represent a **undirected graph** that contains | V | vertices and | E | edges using **adjacency matrix**, you've to create a **2D-Array** of size | V | **x** | V | such that you assign the value **one** to the cells at the **(i, j)**th index and **(j, i)**th index of the **matrix** if there **exists an edge** between the vertices of the graph labelled as **i** and **j** resp.



Graph Operations

Assume you are given an graph that contains | V | vertices and | E | edges.

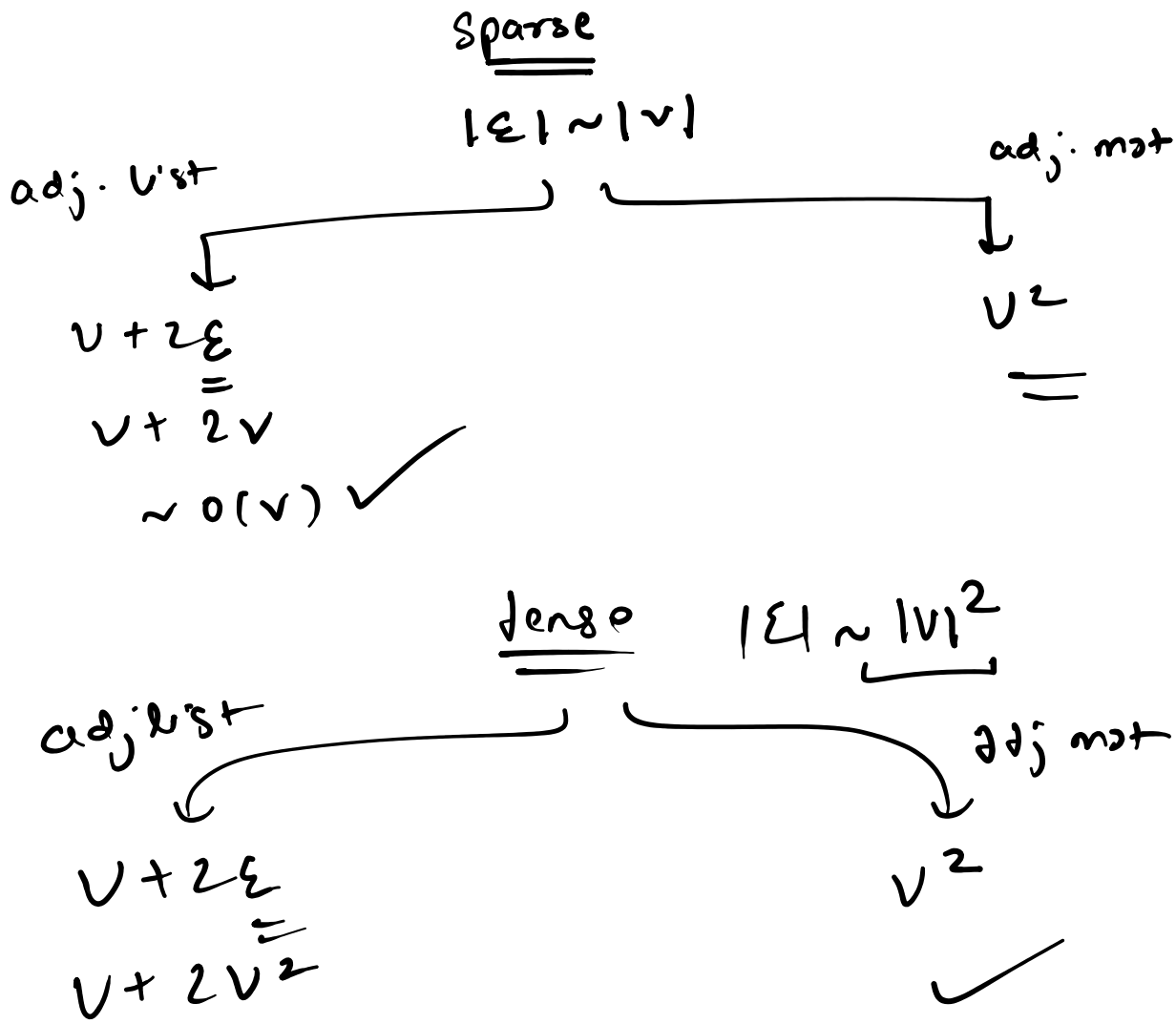
- **Add an edge** between vertex **'u'** and **'v'**
- **Check** if an edge exists between vertex **'u'** and vertex **'v'**
- **Delete** an edge between vertex **'u'** and **'v'**
- List all the **neighbors** of a vertex **'u'**

Adjacency List vs Adjacency Matrix

Adjacency List vs Adjacency Matrix

If the graph is **sparse** i.e. it has **less number of edges** then one should represent it using **adjacency list**. In contrast, if the graph is **dense** i.e. it has **high number of edges** then one should represent it using **adjacency matrix**.

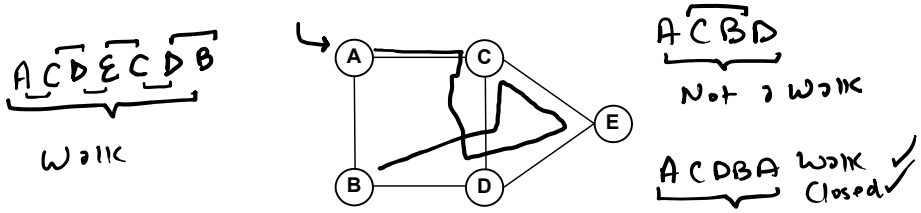
We know that, for a graph that contains **| V |** number of vertices and **| E |** edges the space required to represent it using **adjacency list** is **O(| V | + | E |)** and the space required to represent it using **adjacency matrix** is **O (| V |^2).**



Graph Terminology

- **Walk**

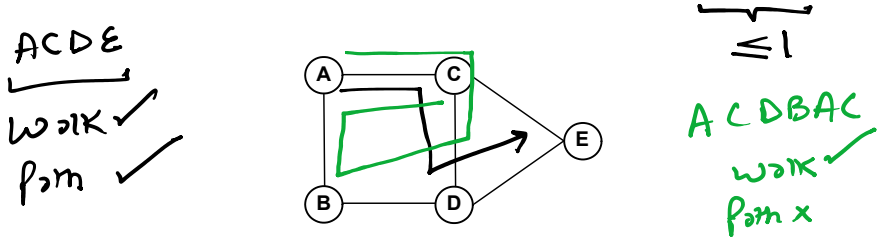
A **walk** in a graph is a **sequence of vertices** such that any two vertices adjacent in the vertex sequence are also adjacent in the graph.



Moreover, we say that a walk is **closed** is it starts and ends at the same graph vertex.

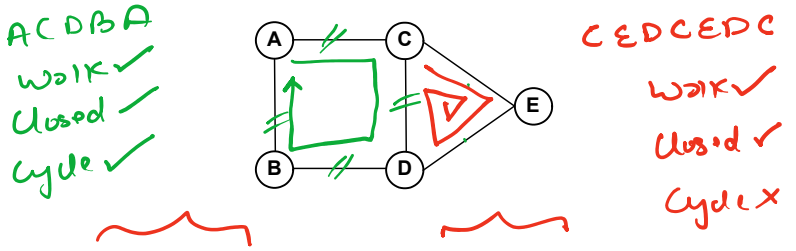
- **Path**

A **path** in a graph is a **walk** in which we visit each vertex of the graph **at most once**.



- **Cycle**

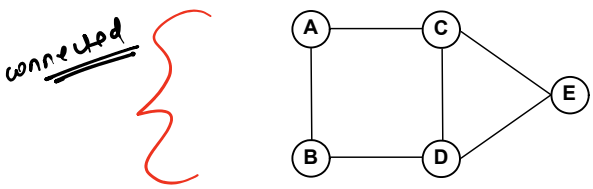
A **cycle** in a graph is a **closed walk** in which we traverse each graph edge **at most once**.



A graph **without any cycle** is also known as an **acyclic graph**.

- **Connected-Graph**

A graph is **connected** if there **exists a path** between **each pair** of graph vertices.

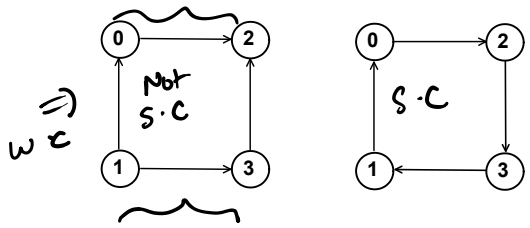


A graph is which is not **connected** is also known as a **disconnected graph.**

⇒ Kosaraju Algo. S.C.C.

- **Strongly-Connected Graph**

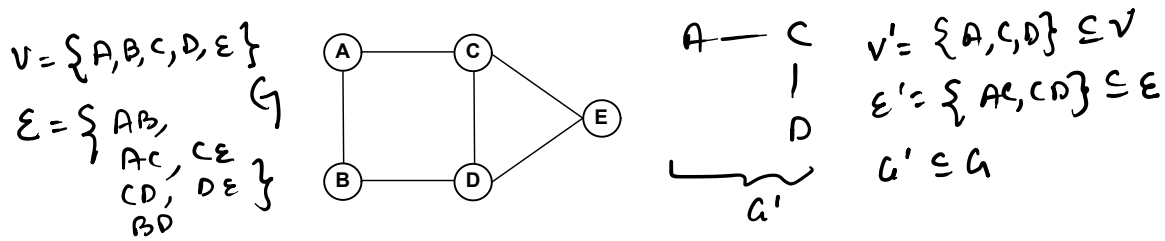
A **directed graph** is **strongly-connected** if there **exists a directed path** between **each pair** of vertices in the graph.



A **directed graph** is **weakly-connected** if there **exists a undirected path** between **each pair** of vertices in the graph.

- **Sub-Graph**

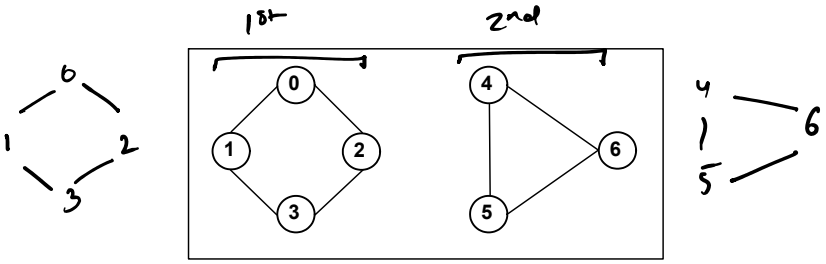
A **sub-graph** of a graph **G = {V, E}** is a graph **G' = {V', E'}** such that **V'** and **E'** are **subsets** of **V** and **E** respectively.



A **proper sub-graph** of a graph is a sub-graph which is the graph itself.

- **Component**

A **component** of a graph is a sub-graph which is **maximally connected**.

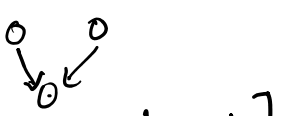


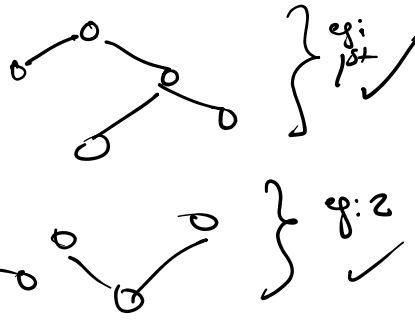
A graph that has **multiple** components is a **disconnected** graph.

Tree =

1. acyclic
2. connected
3. each node has exactly one parent except for root which has no parent

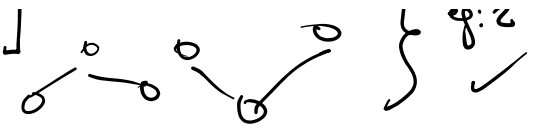
DAG: directed acyclic graph





3. each node has exactly one parent except for root which is w/o par.

acyclic graph



eg: 2

- **Spanning Tree**

A **spanning tree** of a **connected graph** is a sub-graph which is a tree and contains all the vertices of the graph.